

Ejercicio de refuerzo - Semana 2

Comparación defendible de dos modelos sobre tiendas pareadas

Modalidad	Tiempo sugerido	Entrega
Individual	2-3 horas	Proyecto comprimido

Objetivo

Construir una comparación estadística reproducible entre dos modelos evaluados sobre las mismas unidades. El ejercicio integra auditoría de pares, diferencias por tienda, efecto, **incertidumbre**, **prueba t pareada**, **referencia exacta** por **cambios de signo**, **bootstrap pareado** e interpretación limitada por la evidencia.

Escenario

Dos modelos de regresión ya entrenados, A y B, pronostican la demanda semanal de las mismas 16 tiendas. El archivo `data/model_errors.csv` contiene el error absoluto de cada modelo, medido en unidades vendidas. Un error menor representa un mejor pronóstico.

```
store_id,error_a,error_b
S01,12.4,10.6
...
S16,10.6,11.5
```

Procedencia y límite: los datos son sintéticos, deterministas y creados para esta actividad. No provienen de tiendas reales y no constituyen evidencia sobre una empresa, mercado o población existente.

Preparación del proyecto

1. Descargar `ejercicio_semana2_student.zip` desde D2L y descomprimirlo.
2. Acceder a la carpeta `ejercicio_semana2_student` y ejecutar `uv sync --group dev`.
3. Inicializar Git y realizar al menos dos commits con mensajes que describan cambios concretos.
4. Completar `src/week2_exercise/analysis.py` y responder individualmente `informe.md`.
5. Antes de entregar, ejecutar `git log --oneline > git_history.txt`.

```
uv sync --group dev
uv run pytest -q
uv run week2-exercise
```

Reglas de interpretación

- No ejecutar una prueba antes de completar la Parte 1 de `informe.md`.
- Toda respuesta debe citar resultados numéricos o elementos concretos de la figura.
- Una diferencia positiva favorece a B porque se define como error de A menos error de B.

- No reducir la conclusión a significativo o no significativo.
- No interpretar el **p-value** como la probabilidad de que la **hipótesis nula** sea verdadera.

Tarea 1 - Completar el contrato de análisis

Responder las preguntas 1 a 10 de `informe.md` antes de calcular resultados. Se deben declarar los modelos, el proceso objetivo propuesto, la métrica y su dirección, la unidad de análisis, la evidencia de emparejamiento, el **estimando**, las hipótesis, el procedimiento principal, los resultados que acompañarán al **p-value** y los límites del diseño.

$$d_i = e_A(i) - e_B(i)$$

$$\Delta = E(d_i)$$

$$H_0: \Delta = 0 \quad H_1: \Delta \neq 0$$

Tarea 2 - Auditar los datos y construir los pares

Completar `load_model_errors`. Validar que existan las columnas `store_id`, `error_a` y `error_b`; que cada identificador sea único y no nulo; y que los errores sean numéricos, finitos y no negativos. No modificar el CSV original.

Completar `build_paired_table`. La salida debe conservar las tres columnas originales y añadir `difference` y `favors`. Verificar los pares mediante `store_id`; comprobar únicamente la longitud de dos vectores no demuestra que correspondan a las mismas tiendas.

store_id	error_a	error_b	difference	favors
S01	...	...	...	...

Tarea 3 - Hacer visible la evidencia por tienda

Completar `plot_differences`. Crear una figura que muestre una observación por tienda, el signo y la magnitud de `difference`, y una referencia visible en cero. Guardar la figura en `output/store_differences.png`. En `informe.md`, identificar cuántas tiendas favorecen a cada modelo y las diferencias de mayor magnitud en ambas direcciones.

Tarea 4 - Calcular el efecto y su incertidumbre

Completar `summarize_differences` para reportar exactamente `n`, `mean`, `sample_sd` y `standard_error`. La desviación debe ser muestral. Interpretar la diferencia media en unidades vendidas y distinguir la dispersión entre tiendas del **error estándar** de la media.

$$SE = \frac{s_d}{\sqrt{n}}$$

Completar `paired_t_analysis` usando `scipy.stats` sobre el vector de diferencias. Construir un **intervalo t bilateral** del 95 % para la media pareada y conservar grados de libertad, estadístico, **p-value** e intervalo.

Tarea 5 - Comparar dos referencias nulas

Usar como procedimiento principal una **prueba t de una muestra** sobre las diferencias, con **alternativa bilateral**. Después completar `exact_sign_flip_test` y enumerar las 2^{16} configuraciones de signos. Contar las medias nulas cuya magnitud sea al menos tan extrema como la observada.

En `informe.md`, explicar el **p-value** de forma condicional a la **hipótesis nula**, el diseño y los supuestos. También explicar por qué la **prueba t** y los **cambios de signo** no son procedimientos intercambiables aunque produzcan resultados numéricos cercanos.

Tarea 6 - Realizar un bootstrap pareado

Completar `paired_bootstrap_ci`. Usar `np.random.default_rng` con la semilla 20260829 y 10 000 remuestras. Remuestrear tiendas completas mediante el vector de diferencias; no remuestrear por separado los errores de A y B. Reportar el intervalo percentil como análisis de sensibilidad aproximado.

```
SEED = 20260829
BOOTSTRAP_RESAMPLES = 10_000
```

Tarea 7 - Ejecutar el pipeline y redactar la conclusión

Ejecutar `uv run week2-exercise`. El programa debe guardar `output/store_differences.csv` y `output/store_differences.png`, además de mostrar el resumen, la **prueba t**, la referencia por **cambios de signo** y el **intervalo bootstrap**.

Completar las preguntas 11 a 17 de `informe.md` y redactar un informe ejecutivo de máximo 150 palabras para la pregunta 18. La conclusión debe incluir unidad, procedencia, métrica, emparejamiento, efecto, **incertidumbre**, procedimiento, **p-value**, sensibilidad, al menos dos limitaciones y una frase explícita sobre lo que no puede concluirse.

Verificación con `tests/test_public.py`

Las pruebas públicas verifican los contratos de programación y los resultados reproducibles. No evalúan automáticamente la calidad de `informe.md`. Ejecute los comandos desde la raíz del proyecto y no modifique `tests/test_public.py`.

Tarea	Comando	Qué verifica	Salida esperada
1	<code>uv run pytest -v -k "load or invalid"</code>	Columnas, identificadores únicos y errores válidos.	3 passed
2	<code>uv run pytest -v -k "paired_table"</code>	Diferencias A-B y dirección favorecida.	1 passed
3-4	<code>uv run pytest -v -k "summary or t_analysis"</code>	Efecto, dispersión, error estándar , prueba t e intervalo.	2 passed
5	<code>uv run pytest -v -k "sign_flip"</code>	Enumeración exacta y p-value bilateral.	1 passed
6	<code>uv run pytest -v -k "bootstrap"</code>	Remuestreo pareado y reproducibilidad.	1 passed
7	<code>uv run pytest -v -k</code>	Creación de la figura por tienda.	1 passed

Tarea	Comando	Qué verifica	Salida esperada
	"plot"		

Al completar todas las tareas, `uv run pytest -v` debe terminar con 9 passed. Después, `uv run week2-exercise` se debe finalizar sin excepciones y crear los dos archivos de salida.

Entregable

Comprimir el proyecto y subirlo a la carpeta Ejercicios 2 que se encuentra en D2L hasta las 18h30 del martes 1 de septiembre de 2026. La entrega debe contener el código fuente, `pyproject.toml`, `uv.lock`, `git_history.txt`, `informe.md`, el dataset original y los dos archivos generados dentro de `output`. No incluir `.venv`, cachés ni archivos temporales.